

Programming for Analytics

Week 4: Lists, Matrices, Sets, Tuples, Dictionaries, and Reading Minds

Day 4 at The Daily Grind: the order gets a memory, the menu becomes data, and the café reads its customers' reviews.

Romain Claret · University College Dublin, 2026

Monday's practical: a strike from 12:00 to 13:00

A strike on **Monday 5 October**, from 12:00 to 13:00, changes that day's practical. For the tutorial on the week-4 sheet, come in the morning or the afternoon.

Session	Monday 5 October
10:00–12:00	the tutorial on the week-4 sheet
13:00–14:30 *(moved from 12:30)*	**questions only:** the tutorial does not run; the tutors are there to answer your questions
15:00–17:00	the tutorial on the week-4 sheet

Usually at 12:30? Come at 10:00 or at 15:00 this time.

Where you are

Week 4 of 10

You can already: the basics · f-strings · files · functions · `if` · `while` · `for` · UML diagrams

Today you add: lists · slices · **tuples** · **sets** · **dicts** · comprehensions · matrices · text → data (the sentiment lexicon)

Still ahead: memory & unbreakable code (W5) · numpy + pandas + **your project** (W6, a double week) · classes (W7) · functional + GUIs (W8) · complexity (W9) · testing & exam prep (W10)

How last week landed

Self-check 3 closed last night.



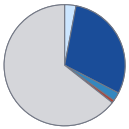
Self-checks taken so far, of 132.
I recommend every one: the exam uses the same formats and similar questions.

Friday's close poll, 47 of you: today's pace



- too slow 9%
- a little slow 0%
- just right 83%
- a little fast 6%
- too fast 2%

All 132 of you: 85 did not answer



- did not answer 85
- too slow 4
- a little slow 0
- just right 39
- a little fast 3
- too fast 1

Only 47 of 132 answered: too few to speak for the room. Answer today's poll, and it can.

Your close poll: asked above everything else: *for and while loops: tracing one line at a time, and how a while loop stops*. The rest of your questions are answered on the **Q&A** page, under Feedback on Brightspace.

Self-check 3, the hardest question (66% right first time):

```
while count ___ 5: # the loop must run until count reaches 5
```

§0 · Outline: what's in this file

§	Section	Café / concept	Page
1	Lists	create, index, slice, grow: the order remembers	8
2	Lists II	modify in place; the dot product = an order total	17
3	Comprehensions & matrices	lists built in one line; lists of lists	20
	Break		23
4	Sets	regulars, unique by construction	24
5	Tuples	the order line `(product, qty)`; immutability	28
6	Dictionaries	**the menu becomes data:** the chain dies	31
7	Case study: sentiment	a lexicon file → a dict → scored reviews	36
8	Exam & exercises · checklist		42

This week's café feature: the order is a **list of tuples** (the receipt lists items again!), the menu and stock are **dicts**, regulars are a **set**, and the café scores its **reviews**. **New concepts:** lists (+comprehensions, matrices), sets, tuples, dicts, and text-as-data.

Where our café is: two debts to pay

Week 3 left us with a register that decides and loops, and **two confessed pains**:

1. *Our order is a single running total. We literally cannot list the items on the bill.*
2. *`price_of()` is an `if/elif` chain that grows by two lines per product.*

Both have the same diagnosis: we've been storing **collections of things** in tools built for **single values**. This week Python hands us four collection types, and both debts die before the break… well, one before, one after. Plus a bonus: the café learns to **read its reviews**.

You asked: a trace runs the loop one line at a time

Self-check 3's hardest question, run by hand. One row per line that runs:

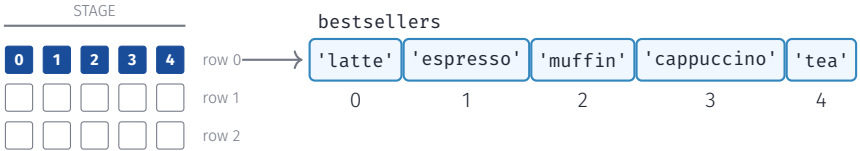
line that runs	`count`	**the check**
`count = 0`	0	
`while count < 5:`	0	`0 < 5` is `True`: into the body
`    count = count + 1`	1	
`while count < 5:`	1	`1 < 5` is `True`
`    count = count + 1`	2	
… three more passes: 3, 4, 5 …		
`while count < 5:`	5	`5 < 5` is `False`: out of the loop
`print(count)`	5	prints `5`

- **A `while` checks its condition only at the top of each pass.** Something in the body must move the condition, here `count = count + 1`. With `<=`, the check at 5 is still `True`, so it prints `6`.
- **A loop that never stops:** press **Ctrl+C** in the terminal, then find the line that should change the condition (week 3's trap #3). `break`, in §1, leaves a loop from inside.
- **Week 3's slide 22 is the same method, shortened:** one row per pass instead of one per line.

For the exam: **run each line in order · write down what changed · stop at the `False`.**

§1 · CONCEPT A list is a row of seats

Look around this auditorium. Lift out the front row, lay it flat, and you have a **list**: each seat holds one value, and the seat's number is its **index**.



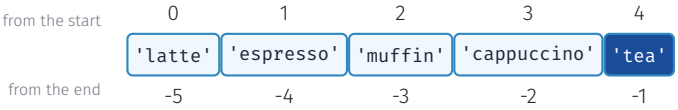
- `bestsellers[2]` asks seat 2 who is sitting there: `'muffin'`.
- A block of seats is a **slice**: `bestsellers[1:3]` is seats 1 and 2, stopping before seat 3.
- A string is a row too, one letter per seat: `"latte"[0]` is `'l'`.

§1 · CONCEPT Lists: sequences of anything

```
>>> bestsellers = ["latte", "espresso", "muffin", "cappuccino", "tea"]
>>> daily_revenue = [212.40, 305.90, 198.20, 240.00, 260.10]
>>> latte_card = ["latte", "Caffè Latte", 3.20, True]     # mixed types are fine
```

- A **list**: values between square brackets, separated by commas: the most powerful everyday object in Python.
- Most languages force one type per list; **Python lists hold anything** (even other lists wait for §3).
- `type(...)` → `<class 'list'>`; `len(bestsellers)` → `5`; `print()` works as always.

§1 · CONCEPT Indexing, and counting from the end



The same row of seats, numbered from both ends: seat 4 and seat -1 are the same seat, the last one.

```
>>> print(bestsellers[0])     # first element: Python counts from ZERO
latte
>>> print(bestsellers[-1])    # count from the END: the last element
tea
```

`bestsellers[-1]` beats the clumsy `bestsellers[len(bestsellers) - 1]` every time. The index is an **offset from the start:** that's why the first one is 0 (Week 3's `range()` logic, landing).

§1 · CONCEPT Slices, changes, deletions

```
>>> print(bestsellers[1:3])    # a SLICE: from 1 up to (not incl.) 3
['espresso', 'muffin']
>>> print(bestsellers[:2])     # from the start
['latte', 'espresso']
>>> print("espresso"[0:3])     # strings slice the same way
esp
>>> print("espresso"[::-1])    # the slice's THIRD slot: a step (-1 walks backwards)
osserpse
>>> daily_revenue[2] *= 1.1      # lists are mutable: change one element in place
>>> del bestsellers[1]          # delete by position (garbage collection collects)
```

And the old trap:

```
>>> print(bestsellers[7])
IndexError: list index out of range
```

Deleting shifts everything after it left: **indexes change under your feet.**

§1 · CONCEPT Growing and shrinking

```
>>> order = []                    # an empty list...
>>> order.append("latte")         # ...grows at the end
>>> order.append("muffin")
>>> last = order.pop()            # pop(): remove the LAST element and hand it back
>>> zeros = [0] * 10              # a list of ten zeros, pre-sized
>>> print("latte" in order)       # membership: the same in as Week 3
True
```

`append()` is the workhorse: you'll type it hundreds of times this term. `pop()` is its inverse. And `in` works on lists as it worked on strings: whole elements this time, not pieces of them.

§1 · CAFÉ The order finally remembers

Debt #1, paid. The order becomes a **list**, one entry per line of the bill:

```
order = []
still_ordering = True
while still_ordering:
    product = input("  Product (or 'done'): ").strip().lower()
    if product == "done":
        still_ordering = False
    elif product not in MENU:
        print("    Sorry, we don't sell that.")
    else:
        quantity = int(input("  Quantity: "))
        if quantity < 1 or stock[product] < quantity:
            print(f"    You can't buy {quantity} {product}(s): {stock[product]} left.")
        else:
            stock[product] = stock[product] - quantity
            order.append((product, quantity))
```

Every item **appends**. Nothing is forgotten. (That `(product, quantity)` pair is a **tuple**, §5; those `MENU` and `stock` lookups are §6. One debt at a time.)

One more loop spelling you'll meet everywhere

```
while True:                # loop forever...
    ...
    if product == "done":
        break              # ...and ESCAPE from inside
```

`while True:` + `break` does the same job as our `still_ordering` flag; the flag *names* the state, `break` jumps ship. Both are correct; read both fluently. (In nested loops, `break` leaves only the **innermost** one.)

§1 · CONCEPT Lists ↔ other iterables

```
>>> list("latte")                    # a string, exploded into characters
['l', 'a', 't', 't', 'e']
>>> list(range(5))                   # a range, materialized
[0, 1, 2, 3, 4]
>>> "flat white, double shot".split(",")
['flat white', ' double shot']
>>> "no sugar please".split()        # default: split on whitespace
['no', 'sugar', 'please']
```

`split()` turns **text into a list of words:** remember Week 2's file lines? Every one of them can be exploded like this. **This is UBER useful:** it's the key to the case study after the break, and to your project's data files.

(And `sys.argv`? It was a list all along: Week 2's promise, kept. Now you can finally slice it: `sys.argv[1:]` is every argument after the program's own name, the pair you'll use forever: `pname, args = sys.argv[0], sys.argv[1:]`.)

§1 · TRY IT Predict, then run (3 min)

```
>>> print(["first", "is", "best"][0])

>>> positions = ["first", "middle", "last"]
>>> print("midpoint:", positions[len(positions) // 2])

>>> print("-".join(["tic", "tac", "toe"]))

>>> seasons = ["Spring", "Summer", "Autumn", "Winter"]
>>> for i, s in enumerate(seasons):
...     print(f"Season {i + 1}: {s}")
```

New friends: `join()` (the inverse of `split()`), and `enumerate()` (index **and** value together, in one loop).

The in-place mutation lesson

Double every element? The obvious loop **silently fails**:

```
>>> numbers = [1, 2, 3, 4]
>>> for n in numbers:
...     n = n * 2
>>> print(numbers)
[1, 2, 3, 4]                # unchanged!
```

What Week 1 showed with `x = int(x)` strikes again: `n = n * 2` builds a **new** object and points `n` at it and **the list never hears about it.** To change the list, assign **at the position**:

```
>>> for i in range(len(numbers)):
...     numbers[i] = numbers[i] * 2
>>> print(numbers)
[2, 4, 6, 8]
```

This distinction (*loop over values* vs *loop over indexes*) is an exam regular.

The dot product IS an order total

Two parallel lists, quantities and prices, position by position:

```
>>> quantities = [2, 0, 4]          # latte, espresso, muffin
>>> prices     = [3.20, 2.50, 2.40]
>>> total = 0.0
>>> for i in range(len(quantities)):
...     total += quantities[i] * prices[i]
>>> print(total)
16.0
```

i	quantities[i]	prices[i]	total
0	2	3.20	6.40
1	0	2.50	6.40
2	4	2.40	16.00

Mathematicians call this a **dot product**. Cafés call it *the bill*. Week 6 will do it in **one line:** remember this slide when it happens.

§2 · CONCEPT The list toolbox

```
>>> queue = ["bob", "ada", "ada"]    # two Adas waiting
>>> queue.reverse()                  # flip in place
>>> daily_revenue.sort()             # ascending, in place
>>> ["latte", 3.20].sort()           # TypeError: '<' between 'float' and 'str'
>>> queue.insert(0, "vip")           # squeeze in at a position
>>> queue.remove("ada")              # removes the FIRST match only...
>>> while "ada" in queue:            # ...so this is the SAFE remove-all
...     queue.remove("ada")
>>> queue.remove("ada")              # none left now: ValueError: x not in list
>>> queue.count("bob")               # how many times?
>>> [1, 2] + [3]                     # concatenate -> [1, 2, 3]
>>> tune = [1, 2]; tune *= 3         # replicate  -> [1, 2, 1, 2, 1, 2]
>>> sum(daily_revenue)               # add it all up (numbers only!)
```

Two of these raise errors on purpose. Read them; they’re §8’s exam bait.

§3 · CONCEPT List comprehensions: lists in one line

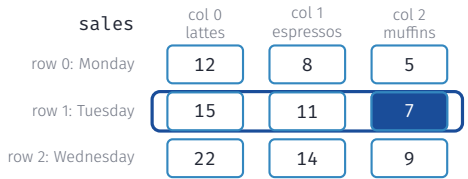
The four canon shapes (all examinable):

```
>>> [x for x in range(5)]                          # build
[0, 1, 2, 3, 4]
>>> [x * 2 for x in range(5)]                      # transform
[0, 2, 4, 6, 8]
>>> [x for x in range(5) if x % 2 == 0]            # FILTER (if at the END)
[0, 2, 4]
>>> ["even" if x % 2 == 0 else "odd" for x in range(5)]   # CHOOSE (if at the START)
['even', 'odd', 'even', 'odd', 'even']
```

- `if` **after** the `for` → decides *which* elements get in.
- `if/else` **before** the `for` is a **ternary** (`"even" if x % 2 == 0 else "odd"` is a value on its own) → it decides *what* each element becomes. Mix both freely.
- One of Python's most-loved features, and Week 8 builds a whole paradigm on it.

§3 · CONCEPT A matrix is the whole auditorium

Put the rows back, and the auditorium of rows and seats becomes a grid of rows and columns: a list of rows, a **matrix**. This one is the week's `sales`, a row per day and a column per product. Two numbers find any cell: the row first, then the column.



`sales[1]` → `[15, 11, 7]`: Tuesday, the whole row
`sales[1][2]` → `7`: Tuesday's muffins

Row first, then column, every time. Changing a cell takes both numbers too: `sales[1][2] += 1`.

§3 · CONCEPT Matrices: lists of lists

Lists hold anything, **including lists**. A sales grid (days × products):

```
>>> # columns: 0=lattes, 1=espressos, 2=muffins
>>> sales = [[12,  8, 5],       # Monday
...          [15, 11, 7],       # Tuesday
...          [22, 14, 9]]       # Wednesday
>>> print(sales[1])             # a whole row -> [15, 11, 7]
>>> print(sales[1][2])          # row 1, column 2 -> 7
>>> for row in sales:
...     print(*row)             # starred print: neat rows (Week 3's trick)
>>> for i in range(len(sales)):
...     for j in range(len(sales[i])):
...         sales[i][j] += 1    # to CHANGE cells: indexes, both levels
```

That grid is begging for totals per product, per day, busiest day… hold that thought; **Week 6 (numpy) eats this grid for breakfast.**

Break

10–15 minutes

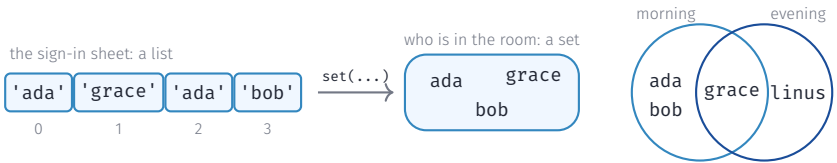
Debt #1 is paid: the order remembers. After the break: ***sets*** *(the regulars),* ***tuples*** *(what those order lines really are), and* ***the dictionary that kills the chain****. Then the café reads minds.*

Think about it: the order lives in a list. The regulars crowd is different: no order, no counting twice. What should the structure that stores them care about?

☎ *Answer on PollEv when we come back.*

§4 · CONCEPT A set is who is in the room

Now forget the seat numbers. A **set** answers one question: who is in the room? Everyone counts once, and nobody has a number.



Two sessions, two sets: `|` everyone · `&` came to both · `-` morning only · `^` exactly one of the two.

- `'ada' in regulars` asks the door: is Ada here? That is what a set is for.
- No seat numbers, so no index: `regulars[0]` is a `TypeError`.

§4 · CONCEPT Sets: unique by construction

```
>>> regulars = {"ada", "grace", "linus"}       # curly braces
>>> vowels = set("aeiou")                      # or set(any iterable)
>>> {"aeiou"}                                  # careful -- braces do NOT explode a string:
{'aeiou'}                                      # that's ONE element, not five
>>> regulars.add("ada")                        # already there -> nothing happens
>>> regulars.remove("linus")
>>> print("ada" in regulars)                   # membership is what a set is for
True
```

- **No duplicates, no order, no indexing:** a set is a bag of *distinct* things.
- Iterable like everything else: `len()`, loops, `*starred` printing all work.
- Made of **immutable** elements (strings ✓, numbers ✓, tuples ✓; lists ✗); the set **itself** is mutable: `add`/`remove` change it in place.
- The full ledger, an exam regular: lists, dicts, sets **mutable**; int, float, str, bool, tuple **immutable**.

Set algebra

```
>>> morning = {"ada", "grace", "bob"}
>>> evening = {"grace", "linus"}
>>> morning | evening        # union:        everyone we saw today
{'ada', 'grace', 'bob', 'linus'}
>>> morning & evening        # intersection: came BOTH times
{'grace'}
>>> morning - evening        # difference:   morning-only crowd
{'ada', 'bob'}
>>> morning ^ evening        # exclusive-or: came exactly once
{'ada', 'bob', 'linus'}
```

*(Run these yourself and the elements may print in another order: sets promise membership, never order. The **contents** are what to check.)*

Four operators, four business questions, for free. And set comprehensions exist too:

```
>>> {w for w in review.split() if w in lexicon}    # which words do we recognize?
```

The regulars

```
    name = input("  Customer name (ENTER to skip): ").strip().lower()
    if name != "":
        if name in regulars:
            print(f"    Welcome back, {name}!")
        else:
            regulars.add(name)
```

One `in`, one `add()`: recognition, with duplicates impossible *by construction*. The day summary reports `len(regulars)` distinct faces.

Should a café remember more than *"seen before?"* A dict of visit counts would, and it unlocks loyalty ideas; a set answers the one question we actually have, and costs nothing to maintain. Pick the structure your **next** question needs. (The counter version is runnable, and on Brightspace with this week's files: `w04_regulars_dict.py`.)

A tuple is a ticket

A **tuple** is a ticket: a few values printed together, in order, for good.

```
ticket = (3, 12)
```



`ticket[0]` → `3` read it ✓

`ticket[1] = 13` → `TypeError` a printed ticket can't be edited ✗

`row, seat = ticket` tear it in two: `row` is 3, `seat` is 12 ✓

- The café's order line `("latte", 2)` is a ticket for the bill: a fact, printed once.
- A ticket can't change, so it goes where a list can't: into a set, and, next section, in front of a dict's colon, as a key.

Tuples: lists that promise not to change

```
>>> line = ("latte", 2)              # parentheses... optional!
>>> line2 = "muffin", 4              # the COMMA makes the tuple
>>> empty = ()
>>> single = "espresso",             # singleton: note the comma!
>>> not_a_tuple = ("espresso")       # <- this is just a STRING in brackets
>>> line[0]                          # ordered + indexable, like a list
'latte'
>>> line[1] = 3                      # TypeError: no modification. Ever.
```

Immutable: no append, no remove, which is exactly why our order lines are tuples: `("latte", 2)` is a fact about the bill, and facts shouldn't wobble. (Bonus: immutability makes tuples **faster**, and legal as set elements & dict keys.)

§5 · TRY IT Tuples & sets, quick fire (3 min)

```
>>> t = ((1, 2, 3), (4, 5, 6))
>>> print((1, 2, 3) in t)            # tuples nest -- predict!

>>> s = {(1, 2, 3), (4, 5, 6)}
>>> s.add((7, 8, 9))                 # tuples CAN live in sets (lists can't) -- why?

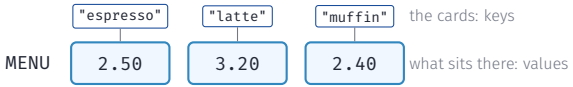
>>> a = (1, 2)
>>> b, c = a                         # UNPACKING (Week 2's multiple returns!)
>>> b, c = c, b                      # the classic one-line swap

>>> print(sorted(set("abcde"), reverse=True))    # sorted() works on any iterable
```

That unpacking line explains the receipt loop you saw in `cafe_w4.py` at the first build: `for product, quantity in order:` unpacks each order line into two names.

§6 · CONCEPT A dict is a row of reserved seats

Last picture: put a name card on each seat. Now you ask for a seat by its card, not its number. That's a **dict**: the card is the **key**, what sits there is the **value**.



a list asks: what sits in seat 2? `MENU[2]` → `2.40`
a dict asks: what sits in the seat marked muffin? `MENU["muffin"]` → `2.40`

- One card per seat: keys are unique, and they never change (strings, numbers, tickets).
- A new card adds a seat at the end; a new value under an old card keeps the seat where it was.
- This room's seating chart is a dict keyed by tickets: `{(3, 12): "ada"}`.

§6 · CONCEPT Dictionaries: lists with meaningful indexes

A list answers *“what’s at position 3?”*. A dict answers *“what’s the price of a latte?”*:

```
>>> prices = {"coffee": 2, "water": 1, "muffin": 3}     # key: value pairs
>>> print(prices["muffin"])                             # look up BY KEY
3
>>> prices["tea"] = 2                                   # add: assign to a new key
>>> prices["tea"] = 2.2                                 # replace: assign to an old key
>>> del prices["water"]                                 # remove an entry
>>> print("coffee" in prices)                           # membership tests KEYS
True
```

- Keys: any **immutable**, unique object (strings, numbers, tuples). Values: anything.
- Also: `dict(pi=3.14, e=2.72)` builds string-keyed dicts; comprehensions work (`{x: x**2 for x in (1,2,3)}`).

§6 · CONCEPT Asking a dict about itself

```
>>> prices = {"coffee": 2, "water": 1, "muffin": 3}
>>> print("Items:", *prices.keys())
Items: coffee water muffin
>>> print("Total:", sum(prices.values()))
Total: 6
>>> for product in prices:              # iterating a dict = iterating its keys
...     print(product, prices[product])
```

`keys()`, `values()`, `sum(...)`: a dict is a lookup table that doubles as a queryable little database. *(Yes, that stock-take one-liner, `sum(prices.values())`, is real café management.)*

And read that first line again: `coffee water muffin` is the order we typed them in. A dict **keeps its keys in the order they were first added**, and replacing a value leaves that key exactly where it was. Print a dict and you can predict the order.

§6 · CAFÉ Debt #2 dies: the menu becomes data

Week 3, fifteen lines, growing two per product:

```
def price_of(product):
    if product == "espresso":
        return 2.50
    elif product == "latte":
        return 3.20
    ...
```

Week 4, two lines, **growing zero per product**:

```
MENU = {"espresso": 2.50, "latte": 3.20, "cappuccino": 3.00,
        "croissant": 2.80, "muffin": 2.40, "tea": 2.20}
```

`MENU[product]` replaces the whole function. `product not in MENU` replaces the 0.0 sentinel. **Adding a product is now one line of *data*, not two lines of *code*.** The same trick gives us per-product `stock`. The muffin rush, generalized.

§6 · TRY IT Nested dicts (3 min)

```
>>> clients = {1: {"name": "James Bond", "id": "1"}}
>>> clients[2] = {"name": "John Doe", "id": "2"}
>>> print(clients[1]["name"])        # dict in a dict: chain the keys

>>> clients[1]["id"] = "007"
>>> clients[1]["drink"] = "Martini"  # grow the inner dict
>>> print(clients[1])

>>> print(clients[0])                # predict THIS one
KeyError: 0
```

Values can be dicts (or lists, or…): structure nests as deep as your data needs. And `KeyError` joins your error vocabulary: *asked a dict for a key it doesn't have.*

Case study: what do our customers think?

The ask: *"We have a file of customer reviews. Which products do people love, and which are letting us down?"*

Sentiment analysis: extract a feeling (good/bad, how strongly) from free text. The professional version needs machine learning. The **useful** version needs Week 4:

1. Load a **lexicon:** words with weights (`love: 0.9`, `stale: -0.8`), from a file, into a **dict**;
2. For each review: **split** the text into words, look each word up, **sum the weights**;
3. Average per product, report.

Decompose → draw it → build. *(Pseudo-code and diagram first; Week 3 habits die hard.)*

§7 · CAFÉ Step 1: the lexicon file → a dict

```
love,0.9
loved,0.9
...
terrible,-0.9
```

```python
DATA_DIR = "data/"

def load_lexicon():
    """Read the sentiment lexicon (word,weight per line) into a dict."""
    lexicon = {}
    for line in open(DATA_DIR + "lexicon.txt"):
        parts = line.strip().split(",")
        lexicon[parts[0]] = float(parts[1])
    return lexicon
```

Week 3's file loop + today's `split()` + a dict = a knowledge base in six lines.
(Weights beat a good/bad coin-flip: "terrible" should hurt more than "dry".)

Step 2: score every review

(We're inside `review_scores()` in `cafe_w4.py`: a `for line in reviews:` loop that has already skipped the file's header line, with empty `totals` and `counts` dicts waiting.)

```
        product, text = line.split(",", 1)
        score = 0.0
        for word in text.lower().split():
            if word in lexicon:
                score = score + lexicon[word]
        if product in totals:
            totals[product] = totals[product] + score
            counts[product] = counts[product] + 1
        else:
            totals[product] = score
            counts[product] = 1
```

- `split(",", 1)` splits **once**: the review text may contain commas itself.
- Unknown words score nothing (`if word in lexicon`); the lexicon is our filter.
- The `in`/`else` at the bottom is the **counting pattern:** you'll use it forever.

§7 · CAFÉ The verdict: run it live

```
>>> print(review_scores(load_lexicon()))   # menu option [4] prints the same, labeled
{'latte': 1.5, 'espresso': 0.53, 'croissant': 1.2,
 'muffin': -0.83, 'cappuccino': 0.7, 'tea': -0.15}
```

(One detail the excerpt hid: the file rounds each average to two decimals with `round(value, 2)` before returning it. That's why you see `0.53` and not `0.5333...`.)

The lattes are beloved. **The muffins are a problem.** That's free text turned into a business decision, made with a dict, a loop and `split()`.

Honesty box: bag-of-words scoring is crude: *"not great"* still scores +0.6 (it saw "great", missed the "not"). Know your tool's limits and say them out loud.

And how would we *know* the lexicon works? The standard move: score texts whose verdict you already know, and **count the hits: an accuracy check**. Our café reviews carry no ready-made labels, so we read the output with judgment instead. But keep the move: when labeled data exists, *measure* your scorer; don't trust it. (Your project's data has exactly such a label: tomorrow's price.)

A flag for your project: this exact skill (text file → lexicon → scores) is how an agent reads the project's news feed. Today wasn't hypothetical.

Rapid prototyping (and a shortcut)

- What we just did has a name: **rapid prototyping:** a few lines of Python to *test an idea* before anyone commits real money to it. It's one of the language's real strengths, and it's what analysts get hired to do.
- A polish for the counting pattern, when you meet it in the wild (both your free books teach it): `totals.get(product, 0)` returns the value **or a default:** collapsing our four-line `if`/`else` into one. Same meaning, fewer lines; use whichever you can read fastest at 2 a.m.

```
if product in totals:                    # Step 2, the four-line version
    totals[product] = totals[product] + score
else:
    totals[product] = score

totals[product] = totals.get(product, 0) + score    # the .get() one-liner
```

Where the café is now

Runnable snapshot: **`cafe_w4.py`:** menu and stock as **dicts**, orders as **lists of tuples** (the receipt lists every line, aligned: Week 2's `receipt_line` is back), regulars as a **set**, and a sentiment report straight from the reviews file. The loyalty-counter version we discussed is runnable too, if you want to compare: `w04_regulars_dict.py`, with this week's files.

The fragility we *chose* to keep:

> *Type **"three"** when asked for a quantity, and the register **dies on the spot** (`ValueError`). One bad keystroke, one dead program, one queue of angry customers.*

Next week: errors, exceptions, and a register that refuses to die.

Exam pointers

The exam can ask you to: predict list operations (indexing/slicing, `sort` on mixed types, `remove` of a value that has gone); rewrite an index-loop as a comprehension (or back); trace the in-place-mutation trap; distinguish list vs set vs tuple vs dict *and justify the choice*; walk the sentiment loop by hand on three lines of input.

Exercises & self-study

(optional; full map on Brightspace):

- **Self-check 4** → Brightspace tonight: about fifteen anonymous minutes, closes Thursday 23:59. We open next week on whatever the class lost.
- **Practical sheet** → Brightspace: Monday's practical works today's material through the week-4 sheet; solutions after the last session.
- **Project groups:** the deadline is the **Sunday of next week** (end of week 5), and the project launches the Friday after. Enter your five now.
- **py4e** *(free, recommended)*: Ch 6 (strings) + Ch 8 (lists) + Ch 9 (dictionaries) + Ch 10 (tuples): videos + auto-graded exercises, **all doable with this week's tools**.
- **ATBS** *(free alternative)*: Ch 6 (lists) + Ch 7 (dictionaries) + Ch 8 (strings). *Heads-up: Ch 6's references/aliasing/`deepcopy` section is next week's memory story; read it as a preview.* · **LPTHW** *(owners)*: Ex 38 (lists) + Ex 39 (dictionaries).

This week's checklist: tick before the practical

- ☐ I can build, index and slice lists, mutate them in place, and say when a value-loop *doesn't*.
- ☐ I can use `append`/`pop`/`insert`/`remove`/`sort`/`count`, and predict their errors (`IndexError`, `TypeError`, `ValueError`).
- ☐ I can write all four comprehension shapes, and index a matrix on both levels.
- ☐ I can choose between a list, a tuple, a set and a dict, and defend the choice.
- ☐ I can unpack tuples (and swap two variables in one line).
- ☐ I can use set operations (`| & - ^`) on real questions.
- ☐ I can walk a dict (`in`, `keys()`, `values()`) and build one by accumulation.
- ☐ I can turn free text into data: `split()`, a lexicon dict, a scoring loop.

Anything unticked → that section of this deck, then the practical sheet.

Next week: memory, errors and exceptions. The register becomes unkillable.